

1. Understand the assignment

- You must build your own original app, not a tutorial clone or renamed template.
- The app must be complete end to end, with a real AI feature inside it.
- You must submit a public GitHub repository and a working live deployed URL.
- The README acts as your full report, so it must include problem, features, AI prompt, tools, screenshots, and setup steps.

Example / Note

This project uses a real problem from NADRA Quetta Region: daily connectivity complaints from 85 DAUs plus AD support tasks.

2. Select a real problem and define the idea

- **Problem:** it is hard to track which DAU complained, when the disruption started, whether the complaint was genuine, and how many cases were resolved by month end.
- **Secondary problem:** AD tasks like OU transfers, password resets, and account unlocks are also handled manually and are difficult to report.
- **Solution idea:** create one app called NADRA LinkOps AI for complaint logging, reporting, AD support tracking, and AI-powered telecom escalation.

Example / Note

Because the idea comes from your own real work context in Pakistan, it is strong for grading under IDEA and COMPLETION.

3. Build the app screens

- Use `index.html` for layout, `styles.css` for design, and `script.js` for logic.
- Create tabs for Dashboard, Complaints, AD Support, Reports, AI Assistant, and Project Notes.
- Use browser `localStorage` to keep the demo simple and fully deployable without a database.

Example / Note

The app already includes sample complaints from PTCL DSL, Wateen, VSAT, DRS, and 3G/4G links in Quetta-region DAUs like Pishin, Chaman, Zhob, and Khuzdar.

4. Add the AI feature

- **Create a serverless function in `api/ai-draft.js`.**
- **Write your own system prompt so the model knows how to classify issues and draft escalation emails.**
- **Pass complaint context, provider, issue type, and remarks into the AI request.**
- **Return a practical answer with Classification, Severity, Escalation Email, DAU Update, and Next Technical Steps.**

Example / Note

If you do not configure an API key yet, the app still demonstrates a fallback draft so you can test the workflow before deployment.

5. Test locally before publishing

- Open index.html directly or run `python -m http.server 8000`.
- Test adding a complaint, editing it, filtering it, and exporting CSV.
- Test an AD request for OU transfer or password reset.
- Generate the monthly disruption report and try Print/Save PDF from the browser.

Example / Note

Good demo test: select the Khuzdar or Chaman complaint and click AI Draft to generate the escalation email workflow.

6. Publish to GitHub

- **Create a new repository on your own GitHub account and keep it PUBLIC.**
- **Run `git init`, `git add .`, `git commit -m`, `git branch -M main`, `git remote add origin`, and `git push -u origin main`.**
- **Open the repository in incognito mode to verify it does not ask for login.**

Example / Note

Do not commit API keys. Keep them only in environment variables on your hosting platform.

7. Deploy on Vercel

- Import your public repository into Vercel.
- Deploy the project with default settings.
- Add environment variables: `OPENROUTER_API_KEY`, `OPENROUTER_MODEL`, and `PUBLIC_APP_URL`.
- Redeploy and test the live site in incognito mode.

Example / Note

Example model value: `openai/gpt-4.1-mini`. Example `PUBLIC_APP_URL`: `https://your-vercel-project.vercel.app`

8. Final README and submission

- **Update README.md with your real GitHub link and your real deployed live URL.**
- **Make sure README includes: app name, problem solved, features list, AI prompt, tools/services/models, screenshots, and run instructions.**
- **Submit only the public GitHub repository link on the portal before the deadline.**

Example / Note

This project folder already includes a README template, PNG screenshots, a system prompt file, and a submission checklist to help you finish quickly.